PROGRAM 4 `4_sparse_matrix_add.c`

Sparse Matrix Addition

C SOURCE CODE 4_sparse_matrix_add.c

```c
/* Sparse Matrix Addition */

#include <stdio.h>

int main() {
    int rows, cols, non_zero1, non_zero2;
    printf("Enter total number of rows and columns for the matrices: ");
    scanf("%d%d", &rows, &cols);
    printf("Enter number of non-zero elements for Matrix 1: ");
    scanf("%d", &non_zero1);
    printf("Enter number of non-zero elements for Matrix 2: ");
    scanf("%d", &non_zero2);

    int sparse1[non_zero1 + 1][3];
    int sparse2[non_zero2 + 1][3];
    int sum[non_zero1 + non_zero2 + 1][3];

    sparse1[0][0] = rows; sparse1[0][1] = cols; sparse1[0][2] = non_zero1;
    sparse2[0][0] = rows; sparse2[0][1] = cols; sparse2[0][2] = non_zero2;
    sum[0][0] = rows; sum[0][1] = cols;

    printf("\nEnter Triplets for Matrix 1 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero1; i++) {
        scanf("%d%d%d", &sparse1[i][0], &sparse1[i][1], &sparse1[i][2]);
    }

    printf("\nEnter Triplets for Matrix 2 (Row, Column, Value):\n");
    for (int i = 1; i ≤ non_zero2; i++) {
        scanf("%d%d%d", &sparse2[i][0], &sparse2[i][1], &sparse2[i][2]);
    }

    int i = 1, j = 1, k = 1;

    while (i ≤ non_zero1 && j ≤ non_zero2) {
        if (sparse1[i][0] == sparse2[j][0]) {
            if (sparse1[i][1] == sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2] + sparse2[j][2];
                i++; j++; k++;
            }
            else if (sparse1[i][1] < sparse2[j][1]) {
                sum[k][0] = sparse1[i][0];
                sum[k][1] = sparse1[i][1];
                sum[k][2] = sparse1[i][2];
                i++; k++;
            }
            else {
                sum[k][0] = sparse2[j][0];
                sum[k][1] = sparse2[j][1];
                sum[k][2] = sparse2[j][2];
                j++; k++;
            }
        }
        else if (sparse1[i][0] < sparse2[j][0]) {
            sum[k][0] = sparse1[i][0];
            sum[k][1] = sparse1[i][1];
            sum[k][2] = sparse1[i][2];
            i++; k++;
        }
        else {
            sum[k][0] = sparse2[j][0];
            sum[k][1] = sparse2[j][1];
            sum[k][2] = sparse2[j][2];
            j++; k++;
```

```
        }
    }

    while (i ≤ non_zero1) {
        sum[k][0] = sparse1[i][0];
        sum[k][1] = sparse1[i][1];
        sum[k][2] = sparse1[i][2];
        i++; k++;
    }

    while (j ≤ non_zero2) {
        sum[k][0] = sparse2[j][0];
        sum[k][1] = sparse2[j][1];
        sum[k][2] = sparse2[j][2];
        j++; k++;
    }

    sum[0][2] = k - 1;

    printf("\nResultant Sparse Matrix (Triplet Representation):\n");
    printf("Row\tCol\tValue\n");
    for (int m = 0; m < k; m++) {
        printf("%d\t%d\t%d\n", sum[m][0], sum[m][1], sum[m][2]);
    }

    return 0;
}
```

PROGRAM OUTPUT Execution Verification

```
Enter total number of rows and columns for the matrices: 3 3
Enter number of non-zero elements for Matrix 1: 2
Enter number of non-zero elements for Matrix 2: 2

Enter Triplets for Matrix 1 (Row, Column, Value):
0 0 5
1 1 3

Enter Triplets for Matrix 2 (Row, Column, Value):
0 0 2
2 2 4

Resultant Sparse Matrix (Triplet Representation):
Row     Col     Value
3       3       3
0       0       7
1       1       3
2       2       4
```

PROGRAM 11 `11_postfix_evaluation.c`

Evaluation of Postfix Expression Using Stack

C SOURCE CODE 11_postfix_evaluation.c

```c
/* Implementation of Postfix Expression Evaluation */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 50

struct Stack {
    int top;
    int items[MAX];
};

struct Stack* createStack() {
    struct Stack* stack = (struct Stack*)malloc(sizeof(struct Stack));
    stack->top = -1;
    return stack;
}

void push(struct Stack* s, int item) {
    if (s->top >= MAX - 1) {
        printf("Stack overflow\n");
        return;
    }
    s->items[++(s->top)] = item;
}

int pop(struct Stack* s) {
    if (s->top == -1) {
        return -999999;
    }
    return s->items[(s->top)--];
}

int main() {
    struct Stack* stack = createStack();
    char postfix[MAX];

    printf("Enter the postfix expression: ");
    fgets(postfix, MAX, stdin);

    // Strip newline character if present
    size_t len = strlen(postfix);
    if (len > 0 && postfix[len - 1] == '\n') {
        postfix[len - 1] = '\0';
    }

    for (int i = 0; postfix[i]; i++) {
        if (isdigit(postfix[i])) {
            push(stack, postfix[i] - '0'); // Convert char directly to int digit
        } else if (postfix[i] == ' ' || postfix[i] == '\t') {
            continue;
        } else {
            int val2 = pop(stack);
            int val1 = pop(stack);

            if (val1 == -999999 || val2 == -999999) {
                printf("Invalid postfix expression\n");
                free(stack);
                return -1;
            }

            switch (postfix[i]) {
                case '+':
```

```
                    push(stack, val1 + val2);
                    break;
                case '-':
                    push(stack, val1 - val2);
                    break;
                case '*':
                    push(stack, val1 * val2);
                    break;
                case '/':
                    if (val2 == 0) {
                        printf("Division by zero error\n");
                        free(stack);
                        return -1;
                    }
                    push(stack, val1 / val2);
                    break;
                default:
                    printf("Invalid operator: %c\n", postfix[i]);
                    free(stack);
                    return -1;
            }
        }
    }

    printf("Result: %d\n", pop(stack));

    free(stack);
    return 0;
}

```

PROGRAM OUTPUT Execution Verification

```
Enter the postfix expression: 2 3 + 4 5 - *
Result: -5
```